UNIDAD 8 / SEMANAS 12 Y 13

Food Store en la feria

La base de la tablet y el cierre de la capa de datos

Trabajo Práctico N.º 8 · Ciclo lectivo 2026
Fecha de preparación: 6 de noviembre de 2026.

Integrantes	
Gonzalo Sanchez	Roman Avila
Mailen Ortiz	Leo Fachinelli
Maximiliano Mendez	Gabriel D'Anna

Regla de oro: el móvil propone y el servidor dispone. SQLite permite consultar y armar borradores sin conexión. PostgreSQL decide precios, stock y ventas. **SIMULADO_APROBADO no confirma una venta ni reserva stock.**

1.a · Qué datos viajan

Datos	¿Viajan?	Finalidad / autoridad
id, nombre	Sí	Identificar y mostrar el producto; origen PostgreSQL.
precio_centavos, stock	Sí	Copia informativa: precio visto y disponibilidad observada, no una garantía.
disponible, eliminado, fecha_copia	Sí	Filtrar catálogo, conservar lápidas y conocer antigüedad.
descripción extensa, imagen, created_at, categoría	No en este TP	Se reduce el tamaño: no son necesarios para probar el flujo.
usuarios, mails, contraseñas, teléfonos	No	Datos ajenos al catálogo; no se guardan ni se envían a IA.

La caché lleva el mínimo necesario para operar offline. El alias local **operador_demo** es ficticio y no autentica a nadie. La fuente de verdad sigue siendo PostgreSQL; la tablet nunca publica su stock visto como stock real.

Alcance verificable

Se reutilizó el esquema del Food Store del equipo preparado en U6 en una base nueva con cuatro productos y un pedido inventados. La versión final se probó con **PostgreSQL 18.6 y SQLite 3.50.4**. No se modificaron bases ni scripts anteriores. El apunte complementario no fue adjuntado; la vista se adaptó al esquema real y a las reglas del PDF de consignas.

1.b–1.c · Base local e integridad

Tabla STRICT	Diseño y restricciones
producto_cache	ID, nombre, centavos INTEGER >= 0, stock_visto >= 0, booleanos 0/1 y fecha de copia.
pedido_borrador	UUID de texto con formato validado, alias ficticio, fecha y pago limitado a EFECTIVO/TARJETA/TRANSFERENCIA.
item_borrador	PK (id_local, producto_id), FK a borrador y producto, cantidad > 0 y precio visto entero no negativo.
cola_sync	UUID como PK; FK al borrador; cuatro estados permitidos. Un solo pendiente por borrador. Triggers impiden reabrir/borrar resueltas.
snapshot_servidor (auxiliar)	CSV del servidor con precio en centavos y booleanos t/f. No es autoridad de ventas.

Archivos: movil/schema_sqlite.sql empieza con PRAGMA foreign_keys=ON; movil/pruebas_sqlite.sql contiene las transacciones y los errores esperados. Se activa la integridad referencial en cada conexión, antes de iniciar transacciones.

COMMIT y ROLLBACK

La primera transacción inserta el borrador, su ítem y su solicitud: persisten **1 + 1 + 1 filas**. Una segunda operación completa se revierte con ROLLBACK; el UUID terminado en 099 no deja filas. No se usa una secuencia de guardados independientes.

C1 · Catálogo sin servidor

```
SELECT id,nombre,precio_centavos,stock_visto
FROM producto_cache
WHERE disponible=1 AND eliminado=0 ORDER BY id;
```

La consulta se ejecutó únicamente sobre el archivo SQLite y devolvió los cuatro productos de la instantánea inicial. Después de C4 quedan visibles 1, 2 y 4. No necesita abrir una conexión PostgreSQL para leer ese catálogo.

C7 · Datos inválidos

INSERT de cantidad **0**: `CHECK constraint failed: cantidad > 0`. INSERT con producto **999999**: `FOREIGN KEY constraint failed`. Cada prueba usa SAVEPOINT/ROLLBACK TO; ambas dejan **0 filas nuevas**. El ejecutor verifica la clase de error y compara el conteo.

SQLite y las restricciones móviles

SQLite es una base embebida en un archivo; PostgreSQL es un servidor con control central de usuarios y concurrencia. La tablet tiene espacio, batería y conectividad limitados: se copia un catálogo pequeño, se agrupan escrituras en transacciones y se sincroniza por lotes al recuperar señal. Una caché antigua puede permitir un borrador, pero nunca garantiza precio ni disponibilidad actuales.

2 · Instantánea y semáforo

Vista: v_snapshot_movil exporta id, nombre, precio*100 convertido a bigint, stock, disponible, eliminado y fecha UTC. Incluye bajas lógicas. COPY TO STDOUT produce CSV real; la importación sustituye snapshot_servidor y hace UPSERT de la caché en una transacción. Última copia: **2026-11-06T19:09:58.340Z**.

Regla: se evalúan solo PENDIENTE. Un ausente, no disponible, eliminado o sin stock produce rojo (RECHAZADO). Si no hay rojo y cambia un precio, amarillo (CONFLICTO). Si todo coincide, verde (SIMULADO_APROBADO). Un borrador vacío se rechaza.

Caso / comando de preparación	Resultado observado y explicación
C2: UPDATE producto SET precio=precio+10 WHERE id=1;	CONFLICTO. El borrador vio 15000 centavos y el servidor pasa a 16000. Aceptación explícita: UPDATE del precio visto + INSERT de solicitud con UUID nuevo en una transacción; da SIMULADO_APROBADO y conserva el conflicto anterior.
C3: borrador de 5 unidades del producto 2; UPDATE producto SET stock=2 WHERE id=2;	RECHAZADO. El servidor solo ofrece 2 unidades; la tablet no corrige ni descuenta stock.
C4: borrador con producto 3; UPDATE producto SET eliminado=true WHERE id=3;	RECHAZADO. Tras recargar, id=3 queda con eliminado=1 y desaparece del catálogo visible, pero la fila sigue referenciada.
C5: borrador con producto 4; nueva exportación sin cambios.	SIMULADO_APROBADO. Coinciden precio, disponibilidad y stock de la instantánea; no se genera pedido de servidor.
C6: INSERT en cola_sync con el mismo solicitud_uuid; repetir semaforo.sql.	Error UNIQUE en cola_sync.solicitud_uuid; 0 filas nuevas. La segunda simulación cambia 0 estados y 0 motivos.

Después de cada cambio de preparación se exportó un CSV nuevo. Los comandos completos de borrador, aceptación y simulación están en ejecutar.py; los CSV por caso y resultados, en ejecuciones/validacion_final/.

Control final y frontera del servidor

```
SELECT count(*) FROM pedido; -- 1 antes y después
SELECT id,stock FROM producto ORDER BY id;
-- (1,50), (2,2), (3,20), (4,15)
```

La reducción de stock de 30 a 2 pertenece a la preparación de C3. Se compararon todos los productos y la cantidad de pedidos **antes y después de cada semáforo**, siempre iguales. En producción, el endpoint autenticado del servidor debe registrar una clave de idempotencia única y confirmar venta/stock en una sola transacción, guardando la respuesta para reintentos. La PK local no reemplaza esa protección.

3.a–3.b · Integración y mantenimiento

Motor	Qué guarda en Food Store	Quién manda
PostgreSQL	Productos, precio, stock, usuarios, pedidos y detalles; reglas e integridad transaccional.	Autoridad comercial y origen del catálogo.
MongoDB · U7	Documentos de pedidos con cliente.id y líneas embebidas; importes Decimal128. Proyección documental del lote de cinco IDs trabajado en U7.	PostgreSQL origina los datos comerciales; MongoDB conserva la proyección documental. No decide stock ni crea una segunda autoridad.
SQLite · U8	Catálogo local, borradores, ítems y cola; instantánea auxiliar para simular.	Autoridad solo del borrador local aún no enviado. El servidor debe revalidar al recibirlo.

La comunicación prevista es tablet → API → PostgreSQL. El transporte CSV sustituye esa API exclusivamente en el laboratorio. MongoDB complementa las consultas documentales; no se implementa una transacción distribuida entre los tres motores.

Plan agregado al mantenimiento de U6

Actividad	Frecuencia y procedimiento propuesto
Respaldo PostgreSQL	Diario a las 00:30 con pg_dump en formato custom; incluir esquema, datos y configuración/permisos por un mecanismo separado. RPO objetivo: 24 h. Suspender escrituras al comparar baseline o compartir snapshot.
Prueba de restauración	Semanal, en base nueva aislada: restaurar, comparar conteos y ejecutar controles de FK, subtotal y suma de detalles. Agregar revisión_integridad.sql; probar una consulta de catálogo y una operación ficticia. RTO objetivo heredado: 10 min, no medido nuevamente aquí.
MongoDB y SQLite	Respaldar documentos propios con mongodump y comprobar mongorestore en otra base. La proyección U7 puede regenerarse desde PostgreSQL. Guardar borradores/cola con backup consistente de SQLite; no copiar a ciegas un archivo activo con WAL. La caché se puede reconstruir.
Servidor restaurado desde respaldo viejo	Pausar sincronización. El servidor anuncia una nueva época de recuperación; invalidar catálogo/versiones en tablets, descargar una instantánea completa y preservar borradores. Reconciliar UUID con el servidor antes de crear nuevas solicitudes. No reenviar verdes como ventas.
Conflictos y rechazados	Operador de turno revisa al reconectar y cada 30 minutos durante la feria; responsable de datos concilia al cierre diario. Precio: pedir aceptación explícita. Stock/baja: informar y modificar o cancelar el borrador; nueva solicitud con UUID distinto.

Disponibilidad y seguridad: respaldos protegidos, acceso mínimo, pruebas fuera de producción y sin credenciales en SQLite. Un stock visto es informativo; dos tablets pueden obtener verde para la misma última unidad, porque este ejercicio no reserva existencias.

3.c · Revisión real con OpenCode Plan

Se enviaron los scripts disponibles de esquema, U3, U4, U5, U6, U7 y U8, sin el archivo de datos personales. OpenCode revisó estáticamente y generó las versiones finales de schema_sqlite.sql y semaforo.sql; ChatGPT/Codex ejecutó después las comprobaciones. La respuesta completa y el adjunto se conservan en docs/evidencias/.

Hallazgo	Decisión	Motivo / evidencia
H1 · U3: falta id en índice cubriente	Aceptado	No prometer Index Only Scan. Medición representativa pendiente.
H2 · U3: subcadena ≠ lexema	Aceptado	agridulce: ILIKE=true, FTS=false. Prueba real en PostgreSQL.
H3 · U3: IDs fijos y umbral >5	Aceptado	Supuestos de carga controlada; proponer RETURNING. No se reejecutó la carga masiva.
H4 · U3/U4: filtros de ventas distintos	Aceptado	Acordar definición de reporte antes de cambiar semántica. Revisión estática.
H5 · U5: mejora de índices no garantizada	Aceptado	Exigir EXPLAIN ANALYZE con carga representativa. No se afirma una medición nueva.
H6 · U6: conteos insuficientes	Corregido	Complemento detecta total incorrecto aunque el conteo no cambie.
H7 · U7: faltantes invisibles	Corregido	LEFT JOIN desde lista esperada detecta IDs ausentes.
H8 · U8: resueltas modificables	Corregido	Triggers de OpenCode impiden reabrir/borrar; nueva validación C1–C7 exitosa.
Desactivar FK y borrar toda la caché	Descartado	Rompe referencias y el contexto offline; DELETE de producto referenciado falla por FK.

Tres decisiones desarrolladas y comprobadas

U6: se cambió temporalmente total de 150 a 151. El conteo siguió en 1, pero el control total vs. suma de detalles devolvió 1 incoherencia. ROLLBACK restauró 150. Se agrega servidor/revision_integridad.sql; no se modifica el TP6 original.

U7: la semilla de U8 tiene deliberadamente solo pedido 1. El control del conjunto esperado {1,2,3,4,5} detectó cuatro faltantes. Esto prueba la mejora ante datos incompletos, no demuestra un fallo del lote original de U7.

Descarte U8: borrar producto_cache.id=1, ya referenciado por un ítem, produjo FOREIGN KEY constraint failed y dejó el registro intacto. No se desactivaron las FK: se mantiene UPSERT y baja lógica. Reabrir una resuelta también falló, ahora por el trigger incorporado.

Los complementos de U6/U7 y las pruebas están en servidor/revision_integridad.sql y pruebas_revision.sql; salida observada: docs/evidencias/revision_postgresql.txt. El historial conserva estado y motivo, pero no todas las versiones de los ítems: una auditoría comercial requeriría ampliar el modelo.

4 · Especificación, IA y defensa

Requisito EARS resumido	Prueba
REQ-01: WHEN no hay conectividad, THE SYSTEM SHALL consultar caché y guardar borrador + ítems + solicitud atómicamente.	C1; COMMIT/ROLLBACK
REQ-02: WHEN falta producto/disponibilidad/stock, THE SYSTEM SHALL rechazar con prioridad sobre precio.	C3, C4; prueba extra rojo + amarillo
REQ-03: WHEN cambia precio sin rojo, THE SYSTEM SHALL marcar CONFLICTO y requerir aceptación con UUID nuevo.	C2
REQ-04: WHEN coincide todo, THE SYSTEM SHALL simular aprobación sin ventas, sin UUID duplicados ni reevaluar resueltas.	C5, C6; integridad C7

Los archivos requirements.md, design.md y tasks.md se redactaron antes de la primera implementación y se solicitaron abrir en Kiro. OpenCode generó después las versiones revisadas del esquema y semáforo a partir del adjunto. No se afirma que el agente de Kiro haya generado código. El equipo debe revisar personalmente la especificación en ese editor.

Bitácora y declaración de IA

La bitácora registra cuatro intervenciones: especificación; implementación inicial y pruebas; revisión y generación con OpenCode Plan; evaluación de hallazgos y nueva ejecución. Se aceptaron restricciones y controles comprobados y se descartó desactivar FK. Se entrega DUIA.md como declaración propuesta: la plantilla oficial no fue adjuntada y debe cotejarse antes de presentar. No se inventan firmas ni se afirma que el equipo ya haya ensayado.

Reproducción y ensayo de 10 minutos

```
cd "C:\Users\Maxi\Desktop\UTN\base de dato\tp8\food-store"
python laboratorio.py --pg-bin "E:\PostgreSQL\bin"
```

El lanzador crea una instancia aislada, corre ejecutar.py y detiene el servidor. Cada corrida produce una carpeta nueva con CSV, base y logs. Si el puerto 55439 está ocupado, indicar --port con otro puerto libre. La validación final del paquete arrancó desde cero y completó C1–C7, pruebas adicionales e integridad sin fallos.

Guion: 0–2 min mapa y autoridad; 2–5 min C2 en vivo; 5–7 min lápida y reintento C6; 7–9 min hallazgo U6; 9–10 min preguntas individuales. Cada integrante debe explicar por qué el verde no es una venta, por qué se usan centavos y dónde iría la idempotencia real.

Archivos y referencias

Entrega: README.md, scripts y base en movil/, scripts PostgreSQL en servidor/, informe y documentos en docs/, evidencias reales en ejecuciones/validacion_final/. La carpeta se entrega comprimida como food-store/.

Fuentes: consigna *BDII_U8_TP8_Food_Store (1).pdf* (6 páginas); esquema y scripts locales del equipo de U3–U7; documentación oficial consultada el 6/11/2026:
SQLite STRICT: https://www.sqlite.org/stricttables.html
SQLite Foreign Keys: https://www.sqlite.org/foreignkeys.html
PostgreSQL COPY: https://www.postgresql.org/docs/18/sql-copy.html